

PADRÕES DE PROJETO · GOF

Factory Method

Criando objetos sem acoplar o código cliente à classe concreta que será instanciada.

Agenda

- 01 Qual problema o Factory Method resolve
- 02 Estrutura UML: papéis e relações
- 03 Exemplo do livro GoF (C++) e demonstração prática (Java, C, Python, Rust)
- 04 Vantagens, cuidados e referências

0 problema

Quando o código cliente decide sozinho qual classe concreta instanciar, **new EmailNotification()**, **new SMSNotification()**... espalhados pelo sistema, qualquer variação nova obriga a alterar código que já funciona.

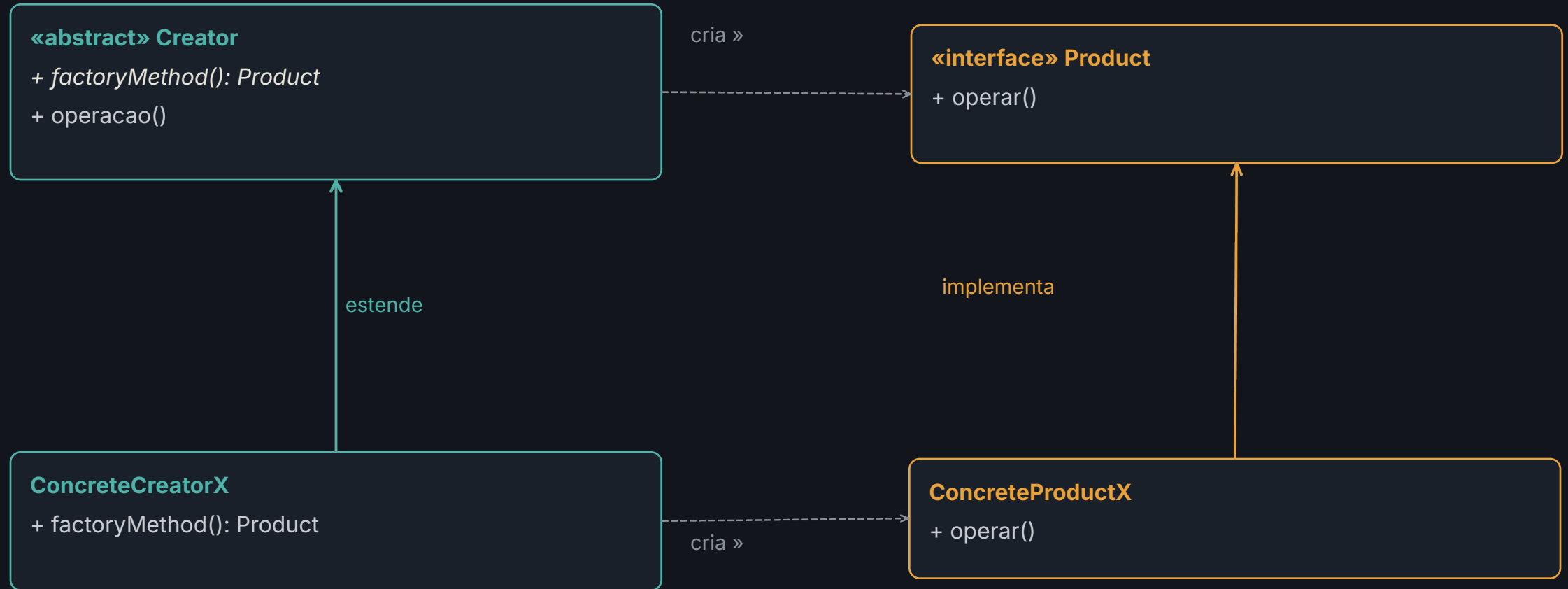
Alto acoplamento

O código cliente conhece e depende diretamente das classes concretas.

Viola o OCP

Adicionar um novo tipo de notificação exige editar um if/else ou switch já existente.

Estrutura UML do padrão



Papéis do padrão

Product

Interface comum dos objetos que a fábrica produz.

ConcreteProduct

Implementação concreta de Product (ex.: EmailNotification).

Creator

Declara o factory method e depende só da interface Product, nunca da classe concreta.

ConcreteCreator

Sobrescreve o factory method para devolver uma instância de ConcreteProduct.

Exemplo na linguagem do livro GoF (C++)

Gamma, Helm, Johnson & Vlissides ilustram os padrões em C++ e Smalltalk.

```
class Creator {
public:
    virtual Product* factoryMethod() const = 0;
    void operacao() const {
        Product* p = factoryMethod();
        p->operar();
        delete p;
    }
};

class ConcreteCreatorA : public Creator {
public:
    Product* factoryMethod() const override {
        return new ConcreteProductA();
    }
};
```

Demonstração prática: notificações

Antes x depois do padrão

✗ Sem Factory Method

```
void enviar(String tipo, String msg) {  
    if (tipo.equals("email"))  
        new EmailNotification().send(msg);  
    else if (tipo.equals("sms"))  
        new SMSNotification().send(msg);  
    // novo tipo → editar aqui de novo  
}
```

Adicionar WhatsApp exige **modificar** essa função.

✓ Com Factory Method

```
void enviar(NotificationFactory factory,  
            String msg) {  
    factory.notify(msg);  
}  
// novo tipo → nada muda aqui
```

Adicionar WhatsApp exige só **criar** WhatsAppNotificationFactory.

Vantagens

Desacoplamento

O cliente conhece só a interface Product, nunca a classe concreta.

Extensível (OCP)

Um tipo novo de produto vira uma nova subclasse, nada existente é alterado.

Fácil de testar

Basta injetar um ConcreteCreator de teste para trocar o comportamento.

Custo: mais uma hierarquia de classes para manter, vale a pena quando novas variações são esperadas.

Referências

- Gamma, Helm, Johnson & Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software* (Addison-Wesley, 1994), capítulo Factory Method.
- Refactoring.Guru, refactoring.guru/design-patterns/factory-method
- SourceMaking, sourcemaking.com/design_patterns/factory_method
- Documentação oficial: Java (docs.oracle.com), Rust Book (doc.rust-lang.org/book), Python (docs.python.org)